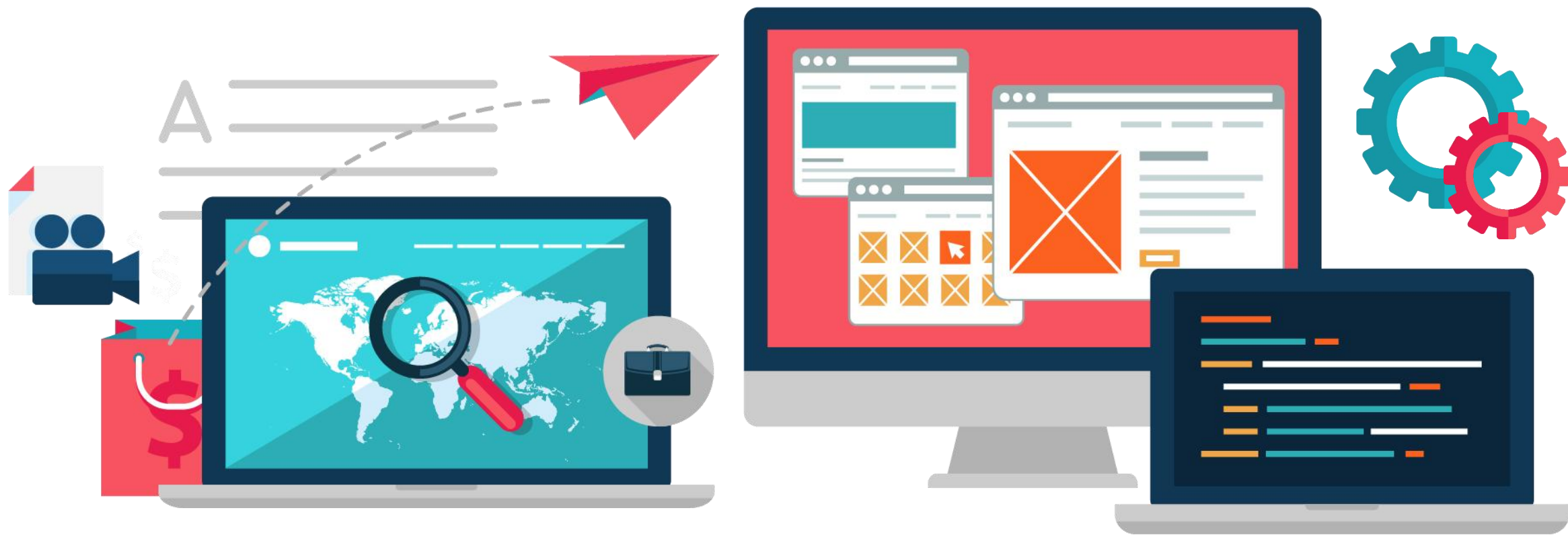


面向对象程序设计

Object Oriented Programming





函数



01

函数声明和定义

02

函数调用过程

03

调用约定

04

函数重载

05

函数的参数

06

函数的返回



- ◆ 函数声明（不同于函数签名/signature）不包含函数体
指明函数的名字、参数个数、参数顺序、参数类型、异常等。
- ◆ 常见格式：
[extern] [调用约定] <返回类型> <函数名> (<参数表>) [const][异常说明];
- ◆ 例：
void Buy(int num); //默认cdecl调用, 源自MSVC官方文档
- ◆
int __cdecl MyFunc(int); //cdecl调用; 形参名可省略
int __stdcall Sell(Sheep & aSheep); //__stdcall调用
int& Max1(int & one,int& two) throw(); //不会抛出任何异常, C++98, C++17移除
int& Max2(int & one,int& two) noexcept; //不会抛出任何异常, C++11
//有利于编译器对程序做更多的优化
- ◆ 返回类型、缺省参数值不能作为区分标志



◆ //返回类型不能作为区分标志

```
int func( ){ }
```

void func(){ }//错误，重复定义

```
int main( ){  
    func( );  
    int n= func( );  
    return 0;  
}
```

◆ //缺省参数不能作为区分标志，

◆ //且编译时匹配

//a.cpp

```
void func(int n=6) { cout<<n; }
```

```
void k( );
```

```
int main( ){
```

```
    func( ); //预编译阶段未改动，编译阶段替换为 func(6)
```

```
    k( );
```

```
    return 0;
```

```
}
```

编译阶段：编译器检查func()的合法性，发现其声明有默认参数n=6，将调用func()改写为func(6)，生成等效的中间代码。

//还有b.cpp

```
void func(int =8);
```

```
void k( ){
```

```
    func(); //预编译阶段未改动，编译阶段替换为 func(8)
```

```
}
```

- ◆ 与变量必须先声明后使用一样，函数也必须在被调用之前先声明。
- ◆ 与变量的声明和定义一样，函数的声明也可以和函数的定义分离。
- ◆ **通常情况下，一个函数可以声明多次但只能定义一次。**
- ◆ 函数应当在头文件中声明，并在源文件中定义：
 1. 把函数声明放在头文件中，可以确保对于指定函数其所有声明保持一致。如果函数接口发生变化，则只要修改其唯一的声明即可；
 2. 定义函数的源文件应包含声明该函数的头文件。这样可使编译器能够检查该函数的定义和声明时是否一致。特别地，如果函数定义和函数声明的形参列表一致，但返回类型不一致，编译器会发出警告或出错信息来指出这种差异。



- ◆ 每个实体（变量、函数、类型等）的**链接性**决定了其名字在不同作用域和编译单元中的**可见性**。
- ◆ 外部链接：名字在多个编译单元中可见，可以被其他文件访问。如全局非静态变量、非静态函数，**显式声明为extern**的常量名，**类、枚举**等。
- ◆ 内部链接：名字仅在当前编译单元内可见，如全局static变量、static函数、未显式声明为extern的常量名、**匿名命名空间内的名字**等。
- ◆ 无链接：名字只在当前作用域内有效，不能在其他作用域引用，如局部变量、函数参数等。
- ◆ 模块链接(C++20起)：这是新引入的，与模块相关。模块内的名字默认具有模块链接，只能在同一个模块内访问，除非导出(export)。



- ◆ 函数调用会有一定的时间和空间开销，特别是对一些函数代码不是很大，但又频繁被调用的函数，会影响程序的执行效率。
- ◆ 通过在编译时将函数体代码插入到函数调用处，将调用函数的方式改为顺序执行方式以节省时间开销，这一过程叫做内联函数的扩展。因此，内联函数实际上是一种用空间换时间的方案。

- ◆ 例，内联函数的定义：

```
inline 函数类型 函数名 (形式参数表)
{
    函数体;
}
```

- ◆ 在内联函数扩展时也进行了实参与形参结合的过程：先用实参名(而不是实参值)，将函数体中的形参处处替换，然后搬到调用处。但从用户的角度看，调用内联函数和一般函数没有任何区别。





- ◆ inline说明对于编译器来说只是一个建议，编译器可以选择忽略这个
- ◆ 一般来说，内联机制适用于优化小的、只有几行的而且经常被调用的函数。大多数的编译器都不支持递归函数的内联。一个 1200 行的函数也不太可能在调用点内联展开。
- ◆ inline函数的定义对编译器而言必须是可见的，以便编译器能够在调用点内联展开该函数的代码。此时，仅有函数声明(即在函数声明前加上inline)是不够的。
- ◆ 把inline函数的定义放在头文件中，可以确保在调用函数时所使用的定义是相同的，并且保证在调用点该函数的定义对编译器可见。
- ◆ 在单个实现文件中，inline函数的定义只能出现一次，在多个实现文件中，inline函数的定义可以出现多次，并且是完全相同的。



◆ 内联函数与带参数的宏定义作用相似，但更安全。

```
#define f(x) x*x
int main( )
{
    int x(2);
    cout<<f(x)<<endl;
    cout<<f(x+1)<<endl;
    return 0;
}
```

程序运行结果：

4

5

原因：

f(x) 替换为 2*2
f(x+1) 替换为 2+1*2+1 }

```
inline int f(int x){ return x*x; }
int main( )
{
    int x(2);
    cout<<f(x)<<endl;
    cout<<f(x+1)<<endl;
    return 0;
}
```

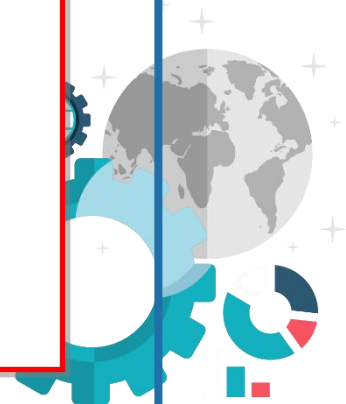
程序运行结果：

4

9

原因：

f(x) 替换为 2*2
f(x+1) 替换为 3*3



```
#include <stdio>
void __cdecl func(int param1, int param2, int param3)
{
    int var1 = param1;
    int var2 = param2;
    int var3 = param3;

    ....
    return ;
}

int main() {
    func(1, 2, 3);
    return 0;
}
```

```
18: int main() {  
..... ; 省略了建立堆栈的代码  
19: func(1, 2, 3);  
00401118 push 3 ; 将 param3 压入栈  
0040111A push 2 ; 将 param2 压入栈  
0040111C push 1 ; 将 param1 压入栈  
0040111E call @ILT+5(func) (0040100a) ; @ILT+5(func) 是函数func的修饰名, 而0040100a则是它的地址  
00401123 add esp,0Ch : 恢复堆栈, __cdecl 规则由调用者 (这里是main) 恢复堆栈  
20: return 0;  
00401126 xor eax,eax  
21: }  
00401128 pop edi  
00401129 pop esi  
0040112A pop ebx  
0040112B add esp,40h  
0040112E cmp ebp,esp  
00401130 call __chkesp (004011d0)  
00401135 mov esp,ebp  
00401137 pop ebp  
00401138 ret
```

从右至左依次压参数入栈

清栈

恢复调用现场

低地址

高地址

...
var3
var2
var1
...
1
2
3

```
3: void __cdecl func(int param1, int param2, int param3) {
```

```
00401020 push    ebp
00401021 mov     ebp,esp
00401023 sub     esp,4Ch
00401026 push    ebx
00401027 push    esi
00401028 push    edi
00401029 lea     edi,[ebp-4Ch]
0040102C mov     ecx,13h
00401031 mov     eax,0CCCCCCCCh
00401036 rep stos dword ptr [edi]
```

记录调用现场

；注意var1, var2, var3 压入堆栈的顺序！

```
4: int var1 = param1;
00401038 mov     eax,dword ptr [ebp+8]
0040103B mov     dword ptr [ebp-4],eax
5: int var2 = param2;
0040103E mov     ecx,dword ptr [ebp+0Ch]
00401041 mov     dword ptr [ebp-8],ecx
6: int var3 = param3;
00401044 mov     edx,dword ptr [ebp+10h]
00401047 mov     dword ptr [ebp-0Ch],edx
```

在栈中建立局部变量

.....；省略了...的代码

```
15: return ;
```

```
16: }
```

```
004010BD pop     edi
004010BE pop     esi
004010BF pop     ebx
004010C0 add     esp,4Ch
004010C3 cmp     ebp,esp
004010C5 call    __chkesp (004011d0)
004010CA mov     esp,ebp
004010CC pop     ebp
```

恢复调用现场

004010CD ret ; 这里是 ret, 由调用者 (main) 恢复堆栈, 但如果是 __stdcall 的话, 恢复堆栈就在这里进行



低地址

高地址



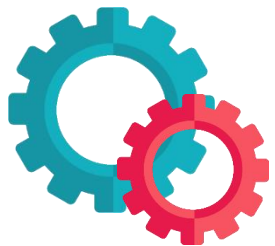
格式:

[extern] [调用约定] <返回类型><函数名> (<参数表>) [const][异常说明];

? 为什么从右到左进行参数压栈?
历史原因

? 谁来负责清栈或恢复堆栈?
调用者负责: 可处理变参和确定参数个数的函数;
被调用者负责: 只能处理确定参数个数的函数;

? 被调用函数的名字如何表示?
同一个项目中的函数名字表示, 限定名不同即可区分;
不同项目中的函数名字表示, 需要区分;





- ◆ 函数调用过程中总是涉及到调用者和被调用者之间参数的传递、内存堆栈的释放处理等细节。
- ◆ **调用约定**(Calling Convention)是指在程序设计语言中为了实现函数调用而建立的一种协议。这种协议**规定了该语言函数调用时的参数传送方式、参数是否可变和由谁来处理堆栈等问题。**
- ◆ x86架构微处理器的调用约定有cdecl、stdcall、fastcall、pascal、thiscall等几种：



◆ cdecl调用约定是源起C语言的一种调用约定，也是C语言的事实上的标准。采用cdecl约定时，函数参数按照从右到左的顺序入栈，并且由调用函数者负责把参数弹出栈以清理堆栈。因此，实现可变参数的函数可以使用cdecl调用约定，如：

◆ `int sprintf(char* buffer,const char* format,...);`

◆ 对应关键字为__cdecl或_cdecl

◆ 编译后的函数名前缀以一个下划线字符



- ◆ stdcall调用约定是由微软创建的，是Windows API的标准调用约定。非微软编译器并不总是支持该调用协议。**stdcall调用约定是cdecl与pascal调用约定的折衷：函数参数按照从右到左的顺序入栈，并且由被调用的函数负责在返回前清理传送参数的栈，函数参数个数固定。**
- ◆ 对应关键字为__stdcall或_stdcall
- ◆ 编译后的函数名后缀以符号"@ "，后跟传递的函数参数所占的栈空间的字节长度。
- ◆ 微软的编译工具规定：PASCAL, WINAPI, APIENTRY, FORTRAN, CALLBACK, STDCALL, __far __pascal, __fortran, __stdcall均是指此种调用约定。





- ◆fastcall调用约定用于对性能要求非常高的场合。通常将函数的从左边开始的两个大小不大于4个字节的参数通过寄存器传递，其余的参数仍旧自右向左压栈传递，并且由被调用的函数负责在返回前清理传送参数的堆栈。
- ◆对应关键字为__fastcall或_fastcall
- ◆fastcall约定还未被标准化，不同编译器的实现也不一致，上述为Microsoft/GCC对fastcall的实现，Borland fastcall则从左至右，传入三个参数至寄存器中，剩下的参数推入栈，也是从左至右



- ◆ 关键字 `__cdecl`、`__stdcall` 和 `__fastcall` 可以直接加在要输出的函数前，也可以在编译环境(微软)的 Setting...->C/C++->Code Generation 项选择。它们对应的命令行参数分别为 `/Gd`、`/Gz` 和 `/Gr`。缺省状态为 `/Gd`，即 `__cdecl`。当加在输出函数前的关键字与编译环境中的选择不同时，直接加在输出函数前的关键字有效
- ◆ 除上面三种常见的调用约定，还有下面三种少见的、被替换的或特有的调用约定：



- ◆ pascal调用约定是基于Pascal语言的调用约定，参数按照从左至右的方式入栈，函数自身清理堆栈，常见在如下16-bit 平台的编译器：OS/2 1.x，Windows 3.x，以及Borland Delphi版本1.x。
- ◆ thiscall调用约定是在调用C++非静态成员函数时使用。对于GCC，thiscall几乎与cdecl等同：调用者清理堆栈，参数从右到左传递，thiscall会在最后把this指针推入栈中，即相当于在函数原型中是隐式的左数第一个参数。对于Visual C++，this指针通过ECX寄存器传递，其余同cdecl约定。



- ◆ `naked` 是一个很少见的调用约定，一般不建议使用。编译器不会给这种函数增加初始化和清理代码，更特殊的是，你不能用 `return` 返回返回值，只能用插入汇编返回结果，此调用约定必须跟 `__declspec` 同时使用。例如定义一个求和程序，如：
- ◆ `__declspec(naked) int add(int a,int b);`
- ◆ x86-64调用约定得益于更多的寄存器可以用来传参，不兼容的调用约定更少，有2种主流的规则：微软x86-64调用约定，System V AMD64 ABI，后者主要在Solaris，GNU/Linux，FreeBSD和其他非微软OS上使用。





- ◆ 什么是名字重整
- ◆ 编译器通常会在生成源程序的目标文件阶段，将其中具有存储性质的对象的名字按照某种既定规则进行改写，以适合以后的编译、链接、汇编使用，这种机制称为名字重整
- ◆ 所谓的具有存储性质的对象，即左值对象，是指要实际占用内存空间、有内存地址的那些实体对象，例如：变量、函数、函数指针等，而数据类型就不属于具有存储性质的对象



◆ C名字重整

- ◆ 名字重整不是一项新技术。Microsoft C编译器在二十世纪八十年代最先引入名字重整，随后各家编译器如Digital Mars, Borland, GNU gcc等纷纷效仿。名字重整还没有统一的规范，各家编译器有独立的名字重整规则。
- ◆ C语言有几种函数调用约定，用某一种调用约定编译的静态库或动态库的函数，如果用另外的调用约定去调用将会导致错误甚至系统崩溃。因此C语言编译器对函数的名字做少量的修饰，用于区别该函数支持哪种调用约定，这可以给编译、链接、特别是库函数的载入提供额外的检查信息。



◆ C++ 名字重整

- ◆ 类似地，C++ 语言存在同样的问题，并且由于含有大量复杂的语言特性，如类、模板、名字空间、操作符重载和函数重载等，使得对象的名字在不同使用上下文中具有不同的意义，从而导致 C++ 语言的名字重整更加必要，也更加复杂。



◆ Microsoft C编译器的名字重整规则简要概括如下：

1. `__cdecl`的函数被改名为 `_name`;
2. `__stdcall`的函数被改名为 `_name@X`;
3. `__fastcall`的函数被改名为 `@name@X`。

X是函数形参所占用的字节长度(包括用寄存器传递的参数)。

例如：

`int __cdecl foo(int i);` 名字重整后 `_foo`

`int __stdcall bar(int j);` 名字重整后 `_bar@4`

`int __fastcall qux(int i);` 名字重整后 `@qux@4`



◆ Microsoft C++ 编译器的名字重整规则简要概括如下：

1. __stdcall

- ① 以 “?” 标识函数名的开始，后跟函数名；
- ② 函数名后面以 “@@yg” 标识参数表的开始，后跟参数表。

参数表以代号表示，代号和参数类型的对应关系是：x - void, d - char, e - unsigned char, f - short, h - int, i - unsigned int, j - long, k - unsigned long, m - float, n - double, _n - bool, ... , pa - 指针，pa后跟代号表明指针类型，如果相同类型的指针连续出现，以 “0” 代替，一个 “0” 代表一次重复。参数表的第一项为函数的返回类型对应的代号，其后依次为各参数类型对应的代号；



◆ Microsoft C++ 编译器的名字重整规则简要概括如下：

1. __stdcall

③参数表后以 “@z” 标识整个名字的结束，如果该函数无参数，则以 “xz” 标识结束。

函数名字重整后，其格式可简单归纳如下：

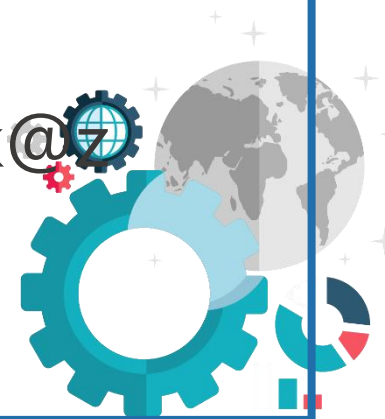
?functionname@@yg***@z //带参数函数

?functionname@@yg*xz //无参数函数

例如：

int test1(char *var1,unsigned long); ?test1@@yghpadk@z

void test2(); ?test2@@ygxxz



◆ Microsoft C++ 编译器的名字重整规则简要概括如下：

2. __cdecl

规则同上面的 __stdcall 调用约定，只是参数表的开始标识由 “@@yg” 变为 “@@ya”。

3. __fastcall

规则同上面的 __stdcall 调用约定，只是参数表的开始标识由 “@@yg” 变为 “@@yi”。

4. __thiscall

由于涉计到类，规则与上面的 __stdcall 调用约定略有不同，参数表的开始标识由 “@@yg” 变为 “@@qae”。



◆ C++ 程序如何调用已经被编译后的C函数

某个C函数的声明: `void foo(int x, int y);`

`foo`编译后在库中的名字为 `_foo`, C++ 编译器在连接时不会去找到 `_foo`, 因此, C++ 程序不能直接调用C函数

◆ C++ 提供了一个 C 连接交换指定符号 `extern "C"` 来解决这个问题。

`extern "C"`

```
{  
    void foo(int x, int y);  
    ... // 其它函数  
    或者#include "myheader.h"  
}
```

这就告诉 C++ 编译译器应该按照C编译器的编译和链接规则来进行链接, 也就是说到库中找名字 `_foo`

C++ 编译器开发商已经对 C 标准库的头文件作了 `extern "C"` 处理, 所以我们可以用 `#include` 直接引用这些头文件。



在**同一作用域**内，多个同名函数，但它们具有不同参数类型、参数顺序、参数个数、const修饰时，可以同时存在，称为函数重载。

◆例：

```
int Func();  
int Func(int);  
int Func(int,int);  
int Func(int,int) const; //只能是成员函数  
int Func(int) throw( );  
int Func(int) throw(int,MyE,YourE);  
int Func(MyClass obj);
```



◆ 为什么函数重载

- ◆ 根据操作数的类型来区分不同的操作，并应用适当的操作，是编译器的责任，而不是程序员的事情

例：int a,b; a+b; string s1,s2; s1+s2;

- ◆ 类似地，程序员可以定义一组函数，它们执行同样的一般性动作，但是应用在不同的形参类型上，调用这些函数时，**由编译器根据所传递的实参类型及个数来判断调用的是哪个函数**

- ◆ 通过省去为函数起名并记住函数名字的麻烦，函数重载简化了程序的实现，使程序更容易理解





- 函数参数的**个数、类型、顺序、const修饰等**不完全相同
- 返回值类型不作为区分标志
- 缺省参数不作为区分标志
- 值类型参数的const型与非const型不作为区分标志
- 引用和指针类型参数，是否可以改变实参，可作为区分标志

//不合法的重载

```
int Fun( );    void Fun( );  
int Fun(int);  int Fun(int=5);  
int Fun(int);  int Fun(const int);  
int Fun(int *); int Fun(int* const);
```

//合法的重载，例

```
void Fun( );  
void Fun( ) const;
```

```
int Fun1(int);  
int Fun1(int *);
```

```
int Fun2(int&);  
int Fun2(const int&);  
int Fun2(int&) throw( );  
int Fun2(int&) noexcept;
```

```
int Fun3(int *);  
int Fun3(const int *);  
int Fun3(int*,int=6);
```

◆ 名字重整:

由编译器在函数名字的基本信息之上, 添加必要的参数信息, 形成新的函数名, 用于区分不同的重载函数。

例: void Fun(int)经重整后, 生成类似_Fun_int的新函数名

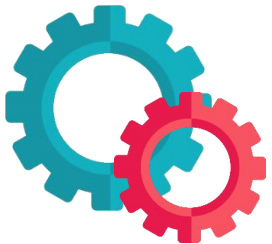
◆ 缺点:

一个程序中难以确定另一个程序中的经重整的函数名

◆ 禁止函数重载:

例:

```
extern "C" {  
    int Fun( );  
    int Fun(int); //非法  
    int OtherFun( );  
}  
extern "C" void MyFunc(int);
```





◆ 函数重载与作用域

- ◆ 重载函数应在同一个作用域中声明。如果局部地声明一个函数，则该函数将屏蔽而不是重载在外层作用域中声明的同名函数

```
void print( const string& );  
void print( double );      //重载  
void foobar( int i )
```

```
{  
    extern void print(int); //隐藏print()    or int print=0;  
    print( "Value" );      //错误  
    print(i);              //正确  
}
```





◆ 重载函数的绑定

◆ 确定候选函数、选择可行函数、寻找最佳匹配

```
void f();
```

```
void f(int);
```

```
void f(int, int);
```

```
void f(double, double = 3.14);
```

```
f(5.6); // calls void f(double, double)
```

```
f(42, 2.56); // error
```

```
f(static_cast<double>(42), 2.56); // calls f(double, double)
```

```
f(42, static_cast<int>(2.56)); // calls f(int, int)
```





- 声明中的参数表（函数原型）：

`void f( );`

`int f(void);`

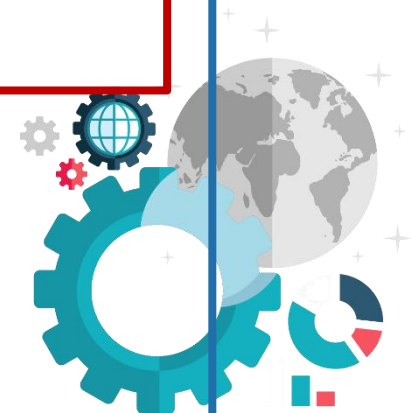
`int& f(int, int& var,const int&,const int *,char='a',int=1);`

`int f(int a, int a);` // 错误：重复参数名

- 定义中的参数表

```
int f( int var,int)
{
    //...
    return var;
}
```

1. 缺省参数，应在函数原型中提供；若无函数原型，才在定义中给出；
2. 定义中的参数，也可以无名





- 函数的缺省参数既可以在函数声明也可以在函数定义中指定默认实参。但是，**在一个文件中，同一作用域内，只能为形参指定默认实参一次。**

```
int ff(int = 0);    // ff.h
```

```
#include "ff.h"    // ff.cpp
```

```
int ff(int i = 0) { /* ... */ } // error
```

- 通常，**应在函数声明中指定默认实参，并将该声明放在合适的头文件中。**
- 如果在函数定义的形参表中提供默认实参，那么只有在包含该函数定义的源文件中调用该函数时，默认实参才是有效的。





- 如果一个函数被声明多次并出现在不同的作用域内，则允许分别为它们指定不同的默认实参

```
int f(int a=6,float b=5.0,char c='.',int d=10);
```

```
int main( )
```

```
{
```

```
    int f(int a=3,float b=2.0,char c='n',int d=20);
```

```
    cout<<f( )<<endl;    //f函数使用局部默认实参
```

```
    return 0;
```

```
}
```



- 压栈顺序由调用约定确定
- 但计算顺序是不确定的
- 缺省参数的匹配
- 实参的类型转换
- 值传递
- 指针传递
- 数组的传递
- 引用传递

```
int a = 2;  
f(++a,++a,++a);  
func( MyF(a),YourF(a)+6 );
```

当一个函数有多个形参时，C++语言没有规定函数调用时实参的求值顺序，编译器根据对代码进行优化的需要自行规定对实参的求值顺序

在实参中注意不要使用带有副作用的运算符，此时可能产生二义性



- 压栈顺序由调用约定确定
- 但计算顺序是不确定的
- **缺省参数的匹配**
- **实参的类型转换**
- 值传递
- 指针传递
- 数组的传递
- 引用传递

缺省参数：

1. 在编译时匹配，而不是运行时；
2. 第一个带缺省值参数的右侧必须都有缺省值；

`void f(int,char= 'c' ,int); //非法`

`void f(int,char= 'c' ,int=9); //正确`

转换：

若不存在完全匹配的函数，尝试类型转换
(每个参数只一次用户定义的类型转换)。

`void f(int);`

`void f(double);`

调用：`f(2); f(2.5); f('a');`

调用：`f(2.5f); //f(double);`



- 压栈顺序由调用约定确定
- 但计算顺序是不确定的
- 缺省参数的匹配
- 实参的类型转换
- **值传递**
- 指针传递
- 数组的传递
- 引用传递

```
int a =10;  const int b=20;  
My mObj;  const My cmObj;  
void f(int ,My);  
或  
void f(const int,const My);
```

```
f(a,mObj); //OK  
f(b,mObj); //OK  
f(a,cmObj); //OK  
f(b,cmObj); //OK
```



- 压栈顺序由调用约定确定
- 但计算顺序是不确定的
- 缺省参数的匹配
- 实参的类型转换
- 值传递
- **指针传递**
- 数组的传递
- 引用传递

```
int a =10; My mObj;  
int * pa =&a; My * pmy=&mObj;  
const int * cpa = pa;  
const My * cpmy = pmy;
```

```
void f(int *,My *);  
void f(const int *,const My *);  
f(pa,pmy); //对应(1)  
f(pa,cpmy); //对应(2)  
f(cpa, pmy); //对应(2)  
f(cpa,cpmy); //对应(2)
```



- 压栈顺序由调用约定确定
- 但计算顺序是不确定的
- 缺省参数的匹配
- 实参的类型转换
- 值传递
- 指针传递
- **数组的传递**
- 引用传递

```
int a[3]={1,2,3};
```

以下几个等价

```
void f(int[ ]);
```

```
void f(int a[ ]);
```

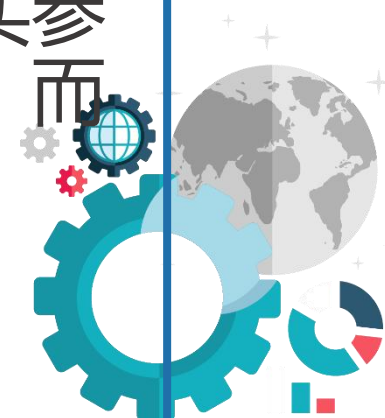
```
void f(int a[3]);
```

```
void f(int a[5]);
```

```
void f(int * a);
```



- 虽然不能直接传递数组，但是函数的形参可以写成数组的形式。虽然形参表示方式不同，但可将使用数组语法定义的形参看作指向数组元素类型的指针。上面的几种定义是等价的，形参类型都是 `int*`
- 通常，将数组形参直接定义为指针要比使用数组语法定义更好。它能明确表示，函数操纵的是指向数组元素的指针，而不是数组本身。由于忽略了数组长度，形参定义中如果包含了数组长度则特别容易引起误解
- 当编译器检查数组形参所关联的实参时，它只会检查实参是不是指针、指针的类型和数组元素的类型是否匹配，而不会检查数组的长度
- `void fun(int array[], int i);` `void fun(int *array, int i);`

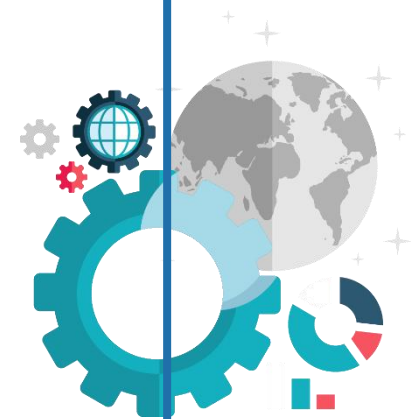


● 二维数组形参

```
void fun(int array[5][10]);    //第一维的大小通常省略  
void fun(int array[][10], int i); //fun(int array[][]);是错误的  
void fun(int (*array)[10], int i);  
void fun(int *array[5], int j);  
void fun(int *array[], int i, int j);  
void fun(int **array, int i, int j);
```



- 每次调用函数时，都会重新创建该函数所有的形参，此时所传递的实参将会初始化对应的形参
- 形参的初始化与变量的初始化一样：如果形参具有**非引用类型**，则复制实参的值，即形参是实参的副本，此时称为**复制传递**、值传递或传值调用，复制传递是单向的，实参值只能传入不能传出，并且在函数内部不能改变实参的值；如果形参为引用类型，则它只是实参的别名，此时称为**引用传递**或引用调用，引用传递是双向的，实参值既能传入也能传出，形参即实参



- 压栈顺序由调用约定确定
- 但计算顺序是不确定的
- 缺省参数的匹配
- 实参的类型转换
- 值传递

- 指针传递

- 数组

- 引用

void f(int, int);

f(a,a);f(a,b);f(a,c),f(a,3)

void f(int&, int&);

f(a,a);**f(a,b); f(a,c);f(a,3);**

void f(int, const int&);

f(a,a);f(a,b); f(a,c);f(a,3);f(b,3);

void f(int&, const int&);

f(a,a);f(a,b); f(a,c);f(a,3);**f(b,b);**

void f(const int&, const int&);

f(a,a);f(a,b); f(a,c);f(a,3);f(b,b);f(3,3);

```
int a=1;
const int & b = a;
const int c=2;
```

● 引用形参对实参的要求

```
int incr(int &val) { return ++val; }  
  
int main( ) {  
    short v1 = 0;  
    const int v2 = 42;  
    int v3 = incr(0);    // error  
    v3 = incr(v2);       // error  
    v3 = incr(v1);       // error  
    v3 = incr(v1 + v2);  // error  
    int v4 = incr(v3);   // ok  
    return 0;  
}
```

不允许传递一个右值或具有需要转换的类型的对象

非const 引用形参只能与完全同类型的非const 对象关联

应该传入一个有效的对象





- 为避免限制对函数的使用，应该将不修改相应实参的形参定义为 const 引用

```
string::size_type find_char(const string &s, char c)
{
    string::size_type i = 0;
    while (i != s.size() && s[i] != c)
        ++i;
    return i;
}
```

```
find_char("Hello World", 'o'); // ok
```

const引用形参更为灵活





- ◆ 函数需显式指定返回类型，**默认返回int型过时且禁止**，`return expr;` `expr`类型和函数返回类型相同，或者能隐式转换为函数的返回类型
- ◆ 函数若无返回值，则必须显式声明为`void`，`return;` 或省略
- ◆ 函数返回类型**不能是函数类型或数组类型**(例如: `int[5]`, `void(int)`), 但可以是它们的指针或引用，或`std::array`、`std::function`等类类型

◆ 按值返回(非引用，非指针)

- 内置类型（基本类型，扩展类型）

`unsigned int f ();` 等价于 `const unsigned int f ();`

- 自定义类型T: **`T f()`不等价于`const T f();`**
- 无论返回内置类型还是自定义类型，返回的都是右值
- 自定义类型的右值，其成员函数仍可被调用，`const`与非`const`不等价，前者只能调用`const`成员函数

◆ 按指针返回

- 空指针 `nullptr` 或非空指针
- 非空指针时需确保是有效指针，即指向的对象有效，不要返回指向局部对象的指针。一旦函数结束，局部对象被释放，返回的指针就变成了指向不再存在的对象的悬垂指针。

```
int *fun(int data)
```

```
{    return &data; } // error
```

```
char *get_str(char str[ ], int x)
```

```
{    return str;    }    // ok
```

- `T * f( );` 等价于 `T * const f( );` 但不等价于 `const T * f( );`

◆ 按引用返回

- $T \ \& \ f()$; 不等价于 $\text{const } T \ \& \ f()$;
- 必须返回一个有效对象的引用

```
int global = 100; //全局变量
```

```
int & other(int &, const int & );
```

```
int & f(int a, int & b, const int & c) { // int *p; return *p;
```

```
    int lVar = 88; //局部临时量
```

```
    static int sVar=99; //局部静态量
```

```
    return a;    return b;    return c;
```

```
    return global; return global+8;
```

```
    return lVar; return sVar; return sVar+2; return sVar+=3;
```

```
    return other(lVar,b); return other(sVar+2,global);
```

```
} // 绿色语句是否合法取决于other的实现, 若other内return lVar;则非法
```



- 返回引用的函数返回一个左值。因此，这样的函数可用于任何要求使用左值的地方

```
char &get_char(char str[ ], int ix)    {    return str[ix];    }  
  
int main( )  
{  
    char s[ ]= "hello" ;  
    get_char(s, 0)= 'H' ;  
    cout << s << endl;  
    return 0;  
}
```





- 返回const型引用
- 当函数返回引用类型时，如果不希望引用返回值被修改，返回类型应该声明为const

```
const char &get_char(char str[ ], int ix)
```

```
{    return str[ix];    }
```

```
get_char(s, 0)= 'H' ; // error
```

同样地，当函数返回指针类型时，如果不希望通过指针返回值修改指针所指向的对象，返回类型也应该声明为const

```
const char *get_char(char str[ ], int ix)
```

```
{    return &str[ix];    }
```

```
*get_char(s, 0)= 'H' ; // error
```



◆ 比较：取两个整数中的最大一个(max2函数)

1. `int max21(int a,int b) { return a>b?a:b; }`
2. `int& max22(int& a,int& b) { return a>b?a:b; }`
3. `const int& max23( int& a, int& b) { return a>b?a:b; }`
4. `const int& max24(const int& a, const int& b) { return a>b?a:b; }`

```
int a =3; int b=4;  
int m1 = max21(a,b);  
cout<<a<<b<<m1<<endl; //输出344
```

```
int m2=max22(a,b);  
cout<<a<<b<<m2<<endl; //输出344
```

```
max22(a,b)=5;  
m2 = max22(a,b);  
cout<<a<<b<<m2<<endl; //输出355
```

```
int m3=max23(a,10);//错误  
max23(a,b)=6;//错误
```

```
int m3 = max23(a,b);  
cout<<a<<b<<m3 <<endl; //输出355
```

```
max24(a,b)=7;//错误  
int m4 = max24(a,b);  
cout<<a<<b<<m4 <<endl; //输出355
```



本章结束